

**SYSTEM VIRTUALIZATION FOR
NEURAL PROCESSING UNITS** 논문의
핵심 기여도(KEY CONTRIBUTION) 및
방법론

목차

- 1. 기존 NPU 방식
- 2. vNPU
- 3. Contribution

기존 NPU 방식

기존 NPU 방식

- 현재 우리는 AI 기반 애플리케이션의 성능을 향상하기 위해, 클라우드 플랫폼은 심층 신경망(DNN)을 위한 신경망 처리 장치(NPU)와 같은 하드웨어 가속기를 채택하고 있다
- NPU(신경망 처리 장치): 딥러닝과 같은 AI 작업을 효율적으로 수행하기 위해 설계된 전용 하드웨어 가속기
- 주요 역할
 1. AI 연산 가속
 2. 효율성

기존 NPU 방식

- NPU 코어의 구성요소

1. Command Processor

2. Direct Memory Access (DMA)

3. Compute Engine

- 3-1. SRAM: HBM과 데이터 교환

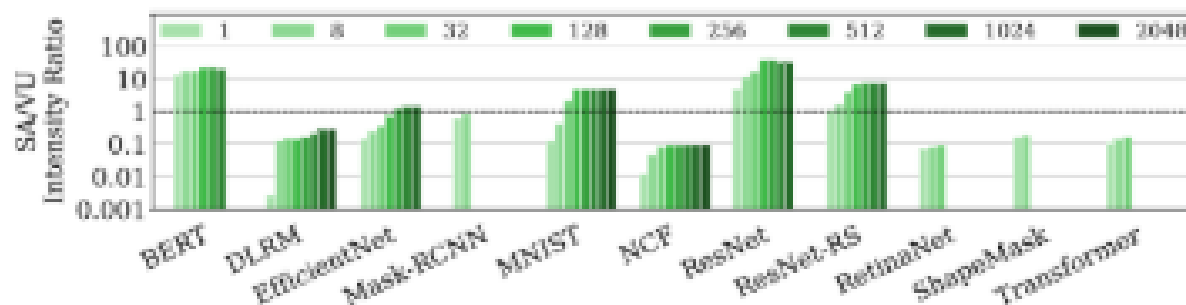
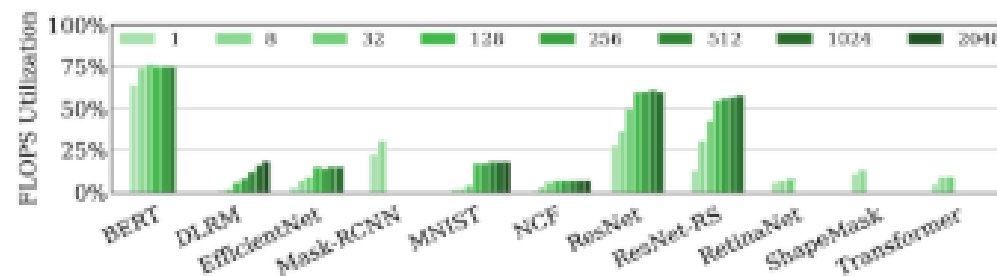
- 3-2. Systolic Arrays (SAs): 행렬 곱셈 및 컨볼루션 연산에서 데이터 재사용과 병렬성을 활용

- 3-3. Vector Units(Vus): 여러 SIMD 유닛을 사용하여 벡터 연산을 수행

기존 NPU 방식

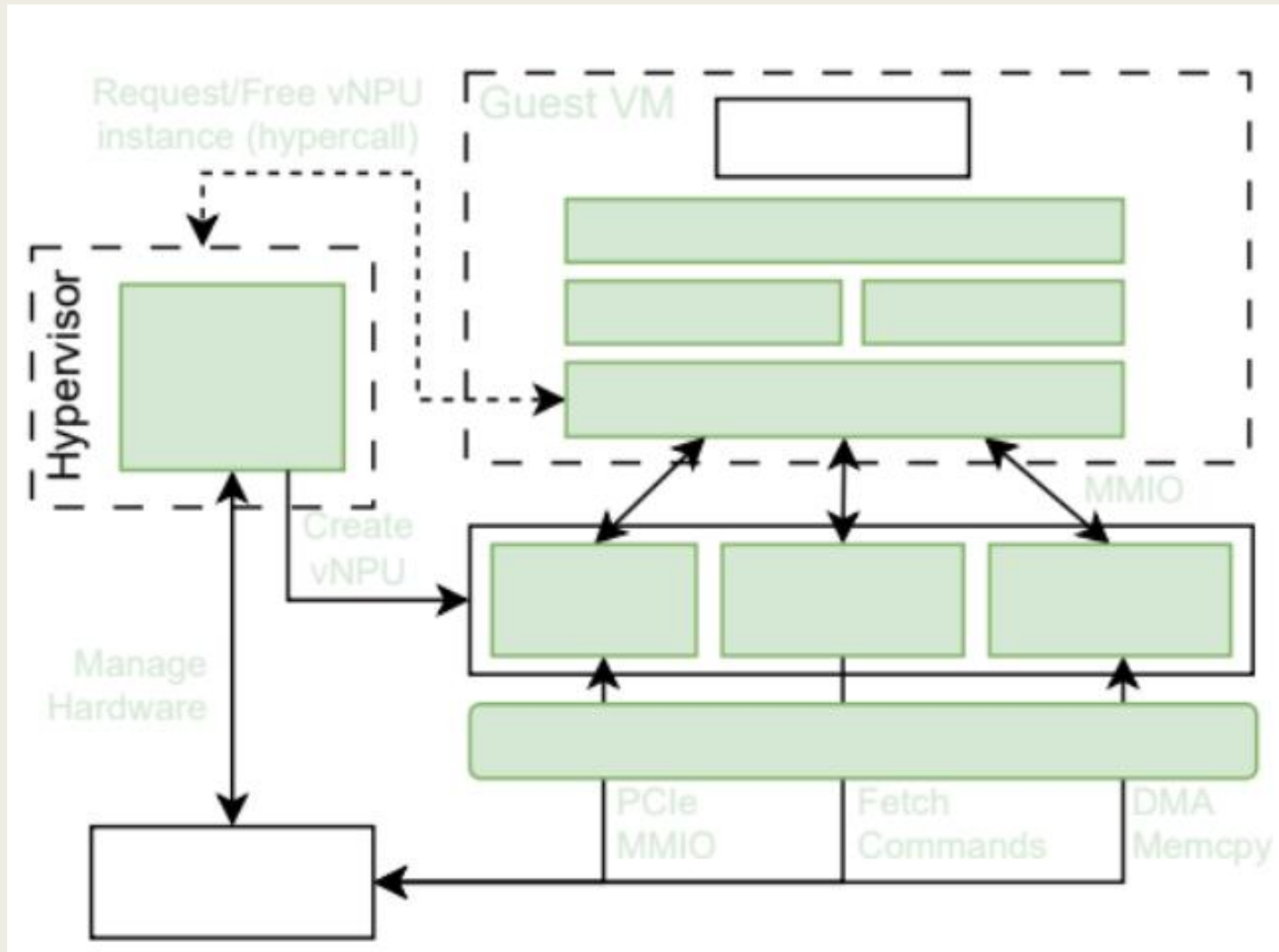
- FLOPS: 컴퓨팅 리소스 활용도
- 각 코어에는 하나의 SA와 하나의 VU가 있음
- 대부분의 ML 워크로드는 TPU 코어 총 FLOPS의 절반 미만을 활용
- 주요 원인: TPU에서 가장 많은 FLOPS를 제공하는 SA가 시간적으로 저활용됨
- ML 워크로드의 성능을 확장하는 일반적인 방법은 NPU 코어 내의 SA 및 VU 수를 늘리는 것
 - > 자원의 낭비
 - > 암달의 법칙에 따라 예견됨
 - => 워크로드에 할당되는 SA와 VU의 수는 수요에 따라 유연 해야함
- Cf) 암달의 법칙: 컴퓨터 시스템의 특정 부분을 개선했을 때, 시스템의 전체 성능이 얼마나 향상될 수 있는지를 계산하는 법칙

DNN Model Names	Category
BERT, Transformer	Natural Language Processing
DLRM, NCF	Recommendation
Mask-RCNN, RetinaNet, ShapeMask	Object Detection & Segmentation
MNIST, ResNet, ResNet-RS, EfficientNet	Image Classification



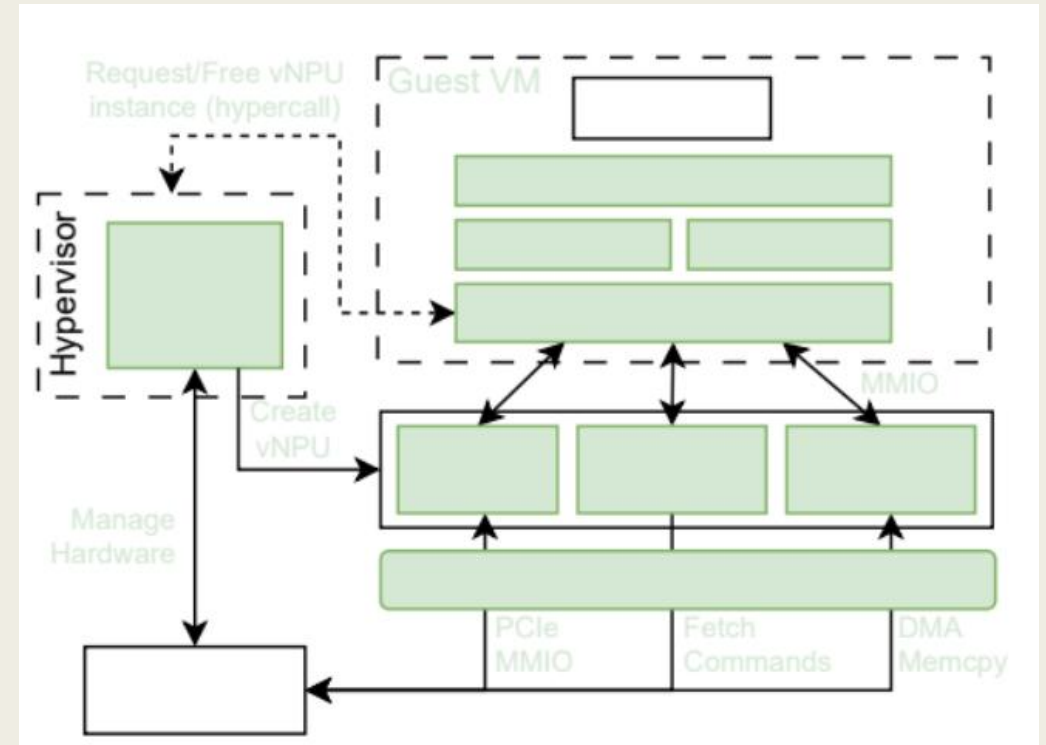
vNPU

vNPU



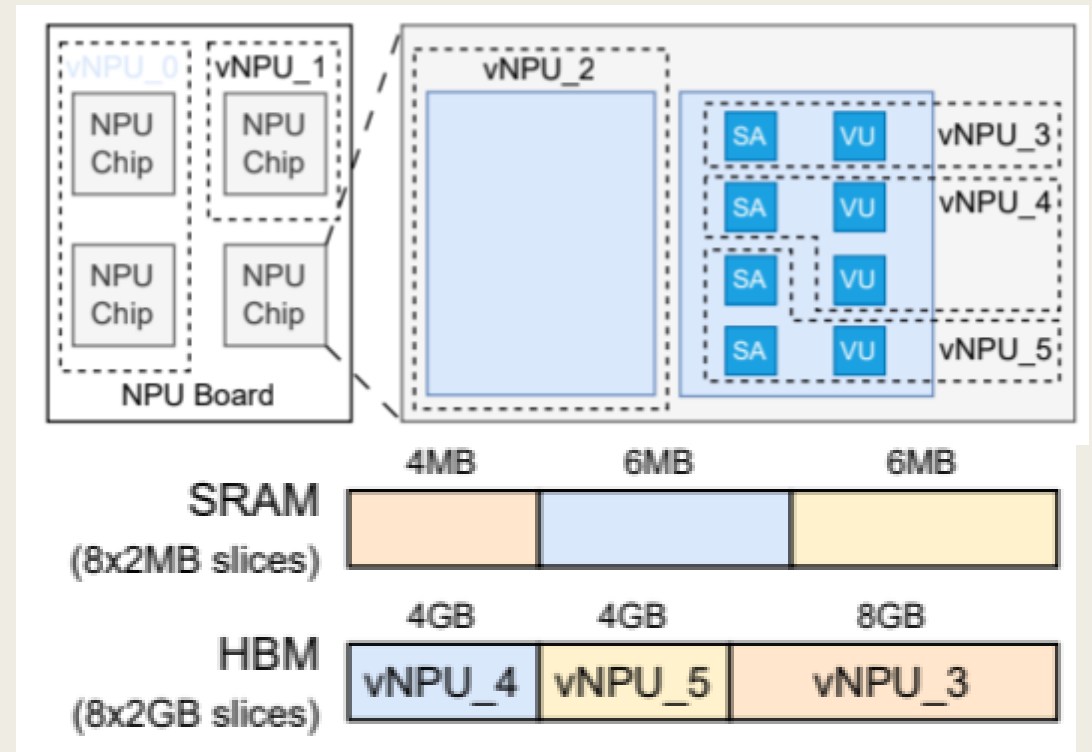
vNPU

- NPU 코어를 가상화 하는데 있어 가장 필수적인 하드웨어 지원: 실행 유닛 단위로 코어를 분할할 수 있는 능력 -> EU는 MIMD(다중 명령 다중 데이터, Multi-Instruction Multi-Data) 을 따라야 함
- NPU용 시스템 가상화 솔루션: NeuCloud



vNPU

- vNPU는 다양한 컴퓨팅 구성을 캡슐화할 수 있으며, 하나의 물리적 NPU 보드는 서로 다른 구성을 가진 여러 vNPU 인스턴스를 호스팅할 수 있음
- 코어 내 SRAM 크기와 HBM 크기를 맞춤 설정 가능



vNPU

- PCIe 장치로 각 vNPU 노출시킴
- vNPU 계층 구조 조회 가능

```
struct vNPU_Board {  
    size_t  num_chips;  
    size_t  num_cores_per_chip;  
    size_t  num_SAs_per_core;  
    size_t  num_VUs_per_core;  
    size_t  sram_size_per_core;  
    size_t  mem_size_per_core;  
}
```

vNPU

- SA와 VU 수의 비율을 결정
- SA에서의 활성 실행 시간: x
VU에서의 활성 실행 시간: y
(이전 VU/SA 연산자의 실행을 기다리는 동안 SA/VU는 유휴 상태)
- 성능 향상 = 개선 전 실행 시간 / 개선 후 실행 시간
- n_x : 사용할 SA 개수
 n_y : 사용할 VU 개수
- 예상가속도(SpeedUp): S

$$S = (x + y) / \left(\frac{x}{n_x} + \frac{y}{n_y} \right)$$

$$U = \frac{S}{S_h} = \frac{(x + y)n_x n_y}{(xn_y + yn_x)(n_x + n_y)}$$

$$U = \frac{k(1 + a)}{(k + a)(k + 1)}, \quad k > 0, a > 0$$

vNPU

- $S_h = n_x + n_y$ (실행 유닛에 관계없는 가상 가속도)
- U : 활용도
- $a = x/y$
- $k = n_x/n_y$
- U 를 최대화하는 k 값 구해야함
- $K = \sqrt{a}$
- 모델별로 최적의 SA와 VU 비율을 계산하여 압달의 법칙에 의한 성능 한계를 극복하고 자원 낭비를 최소화

$$S = (x + y) / \left(\frac{x}{n_x} + \frac{y}{n_y} \right)$$

$$U = \frac{S}{S_h} = \frac{(x + y)n_x n_y}{(x n_y + y n_x)(n_x + n_y)}$$

$$U = \frac{k(1 + a)}{(k + a)(k + 1)}, \quad k > 0, a > 0$$

vNPU

- EU와 메모리 자원 활용도를 모두 최대화하기 위해 물리적 NPU에 최대한 많은 vNPU를 수용하는 것을 목표 -> 모든 vNPU의 총 EU 수가 NPU 코어의 가용 EU 수를 초과하지 않는 범위 내에서 vNPU들을 그룹화 + EU의 수와 메모리 크기의 균형을 맞추려 함
- 동적 재배치(Dynamic remapping): 워크로드 패턴은 시간이 지남에 따라 변할 수 있음
-> NeuCloud 스케줄러는 vNPU 구성을 조정하고 이를 물리적 NPU에 다시 매핑합니다.
- NPU 코어의 서로 다른 EU에 vNPU를 할당한 후에도 오프로드된 워크로드가 없을 때 이러한 vNPU들은 유휴 상태가 될 수 있고, 이는 시간적인 저활용을 야기함
->이 문제를 해결하기 위해 NeuCloud는 코어에 더 많은 vNPU를 매핑함으로써 EU의 과다 할당(oversubscription)을 허용

vNPU

- 여러 애플리케이션 인스턴스가 동일한 NPU를 공유하므로, vNPU 인스턴스 간의 런타임 격리를 강제 -> 보안 격리 + 성능 격리
- 보안 격리: 온칩 SRAM 및 오프칩 HBM에 대한 메모리 주소 공간 격리를 통해 강제
- 성능 격리는 하드웨어와 소프트웨어 기술의 협력을 통해 강제

Contribution

Contribution

- NPU 내부의 물리적 연산 유닛(SA, VU)을 세밀하게 쪼개어 관리할 수 있는 체계를 구축
- 리소스 활용도 극대화
- 성능 및 보안 격리 구현
- 실제적 시스템 통합